



Hojas de Cálculo Eficientes

Arquitectura de Matrices Dispersas en C++

Proyecto de Algoritmos y Estructuras de Datos

2026-1

MOTIVACIÓN Y PROBLEMA

El Desafío de la Escala

Las aplicaciones como Excel manejan millones de celdas potenciales, pero en la realidad, el **99% de ellas suelen estar vacías**.

- Desperdicio masivo de memoria en matrices densas.
- Dificultad para manejar dimensiones dinámicas.
- Lentitud en operaciones sobre datos reales.



¿Por qué Dispersa?

Almacenamos únicamente lo que existe. Si no hay dato, no hay nodo. Optimizamos recursos para lo que realmente importa.

EFICIENCIA DE MEMORIA

99.8%

Ahorro de Memoria

En una hoja de 1000x1000 con 2000 celdas activas.

Matriz Densa



~32.0 MB (Espacio reservado fijo)

Matriz Dispersa



~0.12 MB (Solo nodos existentes)

Memoria Esparcida:

$$M_{\text{total}} = n \times S_{\text{node}} + (R + C) \times S_{\text{header}}$$

02. Diseño de la Estructura

La Lista Enlazada Cruzada (Orthogonal List)

EL NODO CENTRAL

📍 **Coordenadas:** int row, col

💬 **Valor:** std::string value

↔ **Eje Horizontal:** Node* nextInRow

↑ **Eje Vertical:** Node* nextInCol

• NODO DE DATOS (Intersección)

```
struct Node {
    int row, col;
    std::string value;

    // Punteros de enlace cruzado
    Node* nextInRow;
    Node* nextInCol;

    Node(int r, int c, const std::string& v)
        : row(r), col(c), value(v),
          nextInRow(nullptr),
          nextInCol(nullptr) {}
};
```

• CABECERA DE FILA

```
struct RowHeader {
    int row;
    Node* firstCell;
    RowHeader* next;

    RowHeader(int r) : row(r),
                     firstCell(nullptr),
                     next(nullptr) {}
};
```

• CABECERA DE COLUMNA

```
struct ColHeader {
    int col;
    Node* firstCell;
    ColHeader* next;

    ColHeader(int c) : col(c),
                     firstCell(nullptr),
                     next(nullptr) {}
};
```

HEADERS DE FILA Y COLUMNA



RowHeader

Gestiona el acceso a cada fila.

Mantiene un puntero firstCell al inicio de la fila activa.



ColHeader

Gestiona el acceso a cada columna.

Permite recorridos verticales rápidos para operaciones de columna.



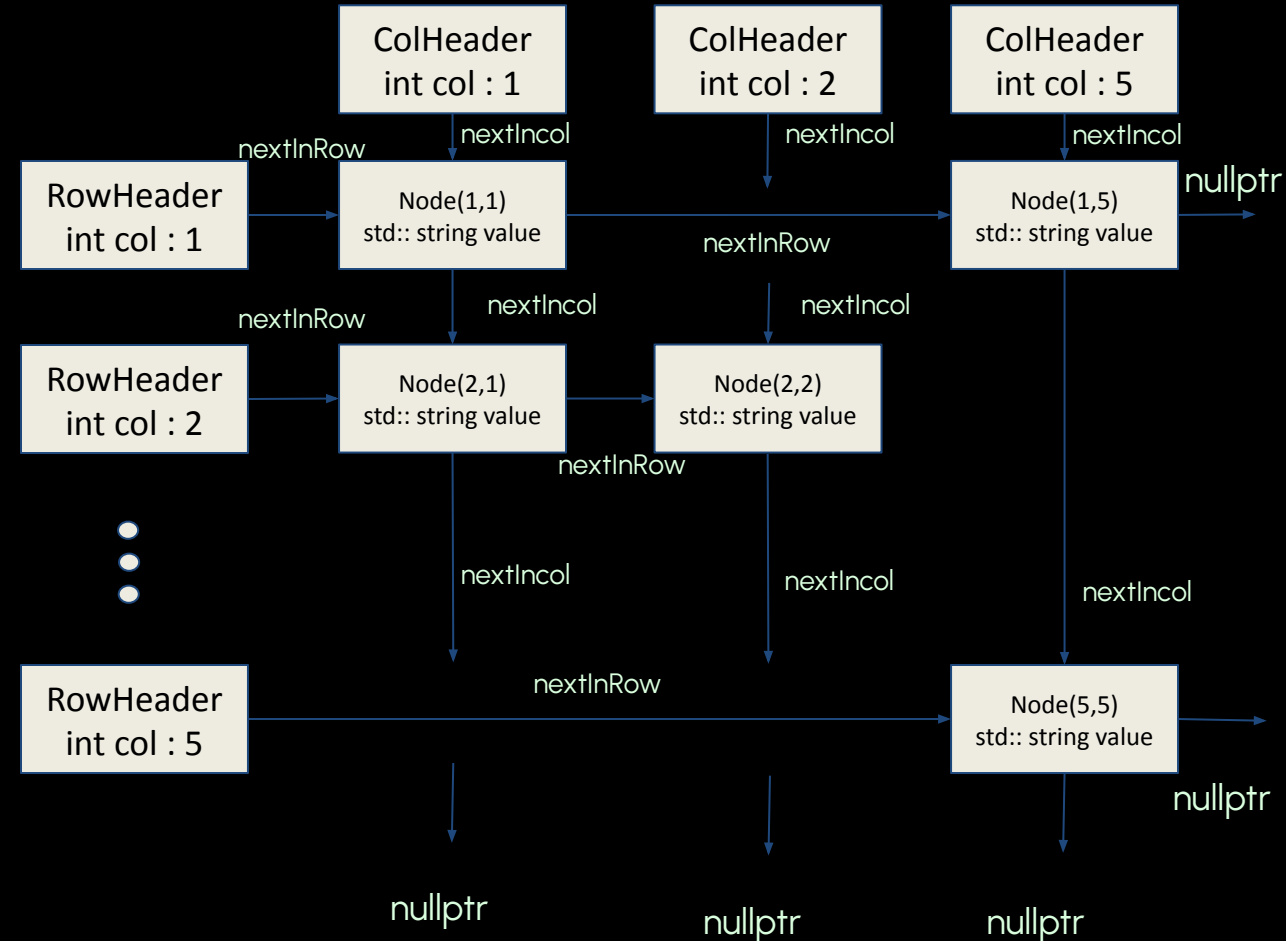
Ordenación

Ambas cabeceras están enlazadas en una lista propia, ordenadas por índice para búsquedas binarias o lineales eficientes.

VISUALIZACIÓN CRUZADA

La matriz se comporta como un tejido: cada nodo pertenece simultáneamente a una lista horizontal y una vertical.

i Las cabeceras actúan como "anclas" que permiten entrar a la red de datos desde cualquier eje.



COMPLEJIDAD TEMPORAL

Operación	Complejidad	Justificación
Búsqueda (findNode)	$O(R + C)$	Localizar Header de fila + buscar en columna.
Inserción (insert)	$O(R + C)$	Búsqueda del punto de inserción en ambos ejes.
Eliminación Celda	$O(R + C)$	Desenlace de punteros previos y siguientes.
Suma de Rango	$O(N_{\text{rango}})$	Recorrido lineal sobre nodos existentes en el área.

RECORRIDO: INSERCIÓN

```
// 2. Enlazar en fila (orden col)
RowHeader* rHead = getOrCreateRowHeader(row);
if (!rHead->firstCell || rHead->firstCell->col > col) {
    newNode->nextInRow = rHead->firstCell;
    rHead->firstCell = newNode;
} else {
    Node* prev = rHead->firstCell;
    while (prev->nextInRow && prev->nextInRow->col < col)
        prev = prev->nextInRow;
    newNode->nextInRow = prev->nextInRow;
    prev->nextInRow = newNode;
}
```

Doble Enlace

El código primero busca o crea la cabecera. Luego, realiza una **inserción ordenada** por índice.

Este proceso se repite para la columna, garantizando la integridad estructural en ambas dimensiones.

RECORRIDO: ELIMINAR CELDA

Actualización de Punteros

Eliminar una celda requiere "coser" la estructura:

- Identificar el nodo objetivo.
- Localizar el nodo prev en la fila.
- Localizar el nodo prev en la columna.
- `prev->next = target->next`

```
// Desenlace de la columna
ColHeader* ch = getColHeader(col);
if (ch->firstCell->row == row) {
    ch->firstCell = target->nextInCol;
} else {
    Node* prev = ch->firstCell;
    while (prev->nextInCol->row != row)
        prev = prev->nextInCol;
    prev->nextInCol = target->nextInCol;
}
```

RECORRIDO: ELIMINAR FILA

```
void SparseMatrix::deleteRow(int row) {  
    RowHeader* rh = getRowHeader(row);  
    if (!rh) return;  
  
    std::vector<int> cols;  
    Node* cur = rh->firstCell;  
    while (cur) {  
        cols.push_back(cur->col);  
        cur = cur->nextInRow;  
    }  
  
    for (int col : cols)  
        deleteCell(row, col);  
}
```

Lógica en Cascada

Para eliminar una fila completa, recorreremos horizontalmente recolectando los índices de columna activos.

Invocamos deleteCell para cada uno, lo que asegura que las columnas verticales también se actualicen correctamente.

RECORRIDO: ELIMINAR COLUMNA

```
void SparseMatrix::deleteCol(int col) {
    ColHeader* ch = getColHeader(col);
    if (!ch) return;

    // 1. Recolectar filas activas
    std::vector<int> rows;
    Node* cur = ch->firstCell;
    while (cur) {
        rows.push_back(cur->row);
        cur = cur->nextInCol;
    }

    // 2. Borrado seguro por celda
    for (int row : rows)
        deleteCell(row, col);
}
```

Lógica Vertical

Para eliminar una columna sin corromper la matriz, primero realizamos un **recorrido vertical** a través de los punteros `nextInCol` para identificar qué filas contienen datos en esa columna específica. Usar un vector temporal evita problemas de punteros colgantes al iterar mientras borramos.

RECORRIDO: ELIMINAR RANGO

```
void SparseMatrix::deleteRange(int r1, int c1, int r2, int c2) {
    // Normalización de índices (Caso Borde)
    if (r1 > r2) std::swap(r1, r2);
    if (c1 > c2) std::swap(c1, c2);

    std::vector<std::pair<int,int>> targets;
    RowHeader* rcur = firstRow;

    while (rcur) {
        if (rcur->row ≥ r1 && rcur->row ≤ r2) {
            Node* cur = rcur->firstCell;
            while (cur && cur->col ≤ c2) {
                if (cur->col ≥ c1) targets.push_back({cur->row, cur->col});
                cur = cur->nextInRow;
            }
        }
        rcur = rcur->next;
    }

    for (auto& [row, col] : targets)
        deleteCell(row, col);
}
```

Normalización con std::swap

El código maneja el error común del usuario al seleccionar un rango en sentido inverso. Al asegurar que $r1 < r2$ y $c1 < c2$, garantizamos que los bucles while funcionen siempre.

Proceso de Dos Pasos

1. **Identificación:** Se recorre la estructura y se guardan las coordenadas en un vector. Esto es eficiente porque solo visitamos nodos que *existen*.
2. **Ejecución:** Se llama a `deleteCell` de forma atómica para limpiar punteros y liberar memoria

DEMOSTRACIÓN DEL PROYECTO

¿Preguntas?

GRACIAS

Repositorio :

https://github.com/Mauricioalarcom/hoja_calculo_simple
